

Recruiter AI Agent

Project Requirements – Prototype to Product

Team Size

2 Members

Duration

8 Weeks

Our Approach

We are planning to build the project in two stages.

Track A will be our prototype, where we will first prove that our AI can actually help a recruiter screen and shortlist candidates.

Once the prototype is working properly, **Track B will become our product version**, where we will improve the system and turn it into a more complete recruitment platform with better UI, multiple users, integrations, analytics, security, and a scalable backend.

So, instead of treating Track A and Track B as two different projects, we will treat them as:

Track A → Prototype → Testing → Track B → Final Product

1. What Problem Are We Solving?

Recruiters often have to go through a large number of resumes for a single job opening. Manually checking every resume takes a lot of time, and it can also be difficult to consistently compare candidates.

Our idea is to build an AI recruitment assistant that can help recruiters with the first stages of hiring.

The system should be able to:

- Understand a job description
- Read candidate resumes
- Identify relevant skills and experience
- Compare candidates with the job requirements
- Give candidates a matching score
- Rank candidates
- Explain why a candidate is a good or weak match
- Help recruiters manage the candidate pipeline
- Help with assessments and interviews

- Provide useful recruitment analytics in the product version

The project document also emphasizes Indian recruitment requirements such as notice periods, salary/ CTC expectations, Indian education qualifications, regional preferences, and Indian resume formats.

2. Track A – Our Prototype

Main Goal

The main goal of Track A is simple:

Can our AI actually help a recruiter find the right candidates from a group of resumes?

We don't want to spend too much time building a complicated system before proving that the main idea works.

So Track A will focus mainly on:

Resume Screening + Job Matching + Candidate Ranking

3. How the Prototype Will Work

The basic flow will be:

Recruiter creates a job

↓

Recruiter uploads resumes

↓

AI reads the resumes

↓

AI understands the job requirements

↓

AI compares candidates with the job

↓

Candidates receive scores



Candidates are ranked



Recruiter reviews the best candidates



Recruiter shortlists candidates



Recruiter schedules an interview

This matches the core Track A requirements in the project document, which include resume screening, candidate ranking, basic interview scheduling, and a candidate dashboard.

4. What We Will Build in Track A

4.1 Job Creation

The recruiter should be able to enter information about the job, such as:

- Job title
- Job description
- Required skills
- Preferred skills
- Required experience
- Education
- Location
- Notice period
- Salary expectations

The system should then convert this information into something the AI can use for matching.

4.2 Resume Upload

The recruiter should be able to upload candidate resumes.

Initially, we will support:

- PDF

- DOCX

We will also try to support multiple resumes at once so that the recruiter doesn't have to upload candidates one by one.

4.3 Resume Understanding

The AI should extract important information from the resume.

For example:

- Candidate name
- Skills
- Education
- Work experience
- Previous companies
- Projects
- Certifications
- Contact details

The idea is that the recruiter shouldn't have to manually read every resume just to collect basic information.

5. Understanding the Job

The system should also understand what the recruiter is actually looking for.

For example:

Job

Python Backend Developer

Requirements

- Python
- FastAPI
- SQL
- REST APIs
- 2+ years experience
- B.Tech/M.Tech
- Maximum 30-day notice period

The AI should convert these requirements into structured information that can be compared with candidate profiles.

6. Candidate Matching

Once the job and resumes are understood, the system will compare them.

For example:

Candidate A

- Python ✓
- FastAPI ✓
- SQL ✓
- REST APIs ✓
- 3 years experience ✓
- B.Tech ✓
- 30-day notice ✓

The system could give:

Match Score: 92%

The exact scoring method will be decided and documented by us during development.

7. Candidate Ranking

Instead of making the recruiter check every candidate individually, the system will rank them.

Example:

Rank	Candidate	Score	Recommendation
1	Candidate A	92%	Strong Match
2	Candidate B	86%	Strong Match
3	Candidate C	76%	Good Match
4	Candidate D	61%	Needs Review

This is important because candidate ranking and screening accuracy are major parts of the Track A evaluation.

8. Explainable AI

We don't want the system to simply say:

"Candidate A = 92%"

We want it to explain the result.

For example:

Candidate A – 92%

Why they are a good match:

- Strong Python experience
- Has FastAPI experience
- Meets the required experience
- Has the required education
- Notice period matches

Possible gap:

- No direct AWS experience

Overall recommendation:

Strong Match

This makes the system easier for a recruiter to trust and understand.

9. Candidate Pipeline

The recruiter should be able to track where each candidate currently stands.

For example:

Applied → Screening → Shortlisted → Interview → Selected / Rejected

The prototype will have a simple candidate pipeline.

The product version will make this much more detailed.

10. Interview Scheduling

Track A will include basic interview scheduling.

The recruiter should be able to:

- Select a candidate
- Choose a date
- Choose a time
- Select interview type

- Save the interview
- View upcoming interviews

If possible, we will connect it with a calendar API.

11. Candidate Communication

The AI can also help generate simple recruiter messages.

For example:

- Interview invitation
- Shortlisting message
- Rejection message
- Follow-up message

The original Track A requirements also include candidate communication templates.

12. Track A Technology

For the prototype, we will keep the technology relatively simple.

Frontend

Streamlit

Backend

Python

AI

LangChain + LLM

NLP

spaCy

Data Processing

pandas

Database

SQLite

Deployment

Streamlit Cloud

These are consistent with the suggested Track A stack in the project document.

13. Track A UI

The prototype will have a simple recruiter-focused interface.

Main pages

Dashboard

Shows:

- Number of candidates
- Shortlisted candidates
- Interviews
- Selected candidates

Jobs

- Create job
- View jobs
- Edit requirements

Resume Screening

- Upload resumes
- Process resumes
- View extracted information

Candidates

- Candidate list
- Candidate profile
- Match score
- Ranking
- Status

Interviews

- Schedule interview
- View interviews

Reports

- Candidate reports
- Basic recruitment statistics

The official Track A requirements call for a Streamlit interface with candidate pipeline tracking, resume upload/batch processing, matching, scheduling, communication, and export functionality.

14. Track A – What We Consider a Successful Prototype

At the end of Track A, we should be able to demonstrate something like:

"Here is a Python Developer job. We uploaded 10 resumes. The system read all of them, identified their skills and experience, compared them with the job requirements, ranked the candidates, and showed us why the top candidates were recommended."

If we can do that reliably, our prototype is successful.

15. Moving From Track A to Track B

After we have the Track A prototype working, we will not throw it away.

Instead, we will take the useful parts and build the product around them.

For example:

Track A

Resume Parser

↓

Matching Algorithm

↓

Candidate Ranking

Track B

These become proper backend services:

Resume Service

↓

Matching Service

↓

Ranking Service

↓

Recruitment Agent

This way, the work we do in Track A becomes the foundation of the final product.

16. Track B – Our Product

Main Goal

Track B is where we turn our working prototype into something that feels like an actual recruitment product.

The main question becomes:

Can a recruiter use our system to manage the complete hiring process instead of only screening resumes?

The official Track B requirements expand the project into a comprehensive talent acquisition platform with advanced workflows, 5+ integrations, LangGraph, professional React/Next.js UI, role-based access, candidate journey tracking, assessments, analytics, and production deployment.

17. Product Workflow

The final product should support something closer to:

Create Job

↓

Find / Add Candidates

↓

AI Screening

↓

Candidate Ranking

↓

Assessment



Interview



Interview Feedback



Hiring Decision



Offer



Analytics

This makes the final system much more than a resume screening tool.

18. Product Users

The product will have three main types of users.

Recruiter

The recruiter can:

- Create jobs
- Add candidates
- Screen candidates
- View AI recommendations
- Shortlist candidates
- Schedule interviews
- Manage candidate status

Hiring Manager

The hiring manager can:

- View shortlisted candidates
- Review candidate profiles
- Give feedback

- Rate candidates
- Participate in hiring decisions

Admin

The admin can:

- Manage users
- Manage roles
- Manage integrations
- View system-level information

Role-based authentication is specifically part of the Track B requirements.

19. Product Dashboard

The final dashboard should feel more like a real HR/recruitment application.

It can show:

- Open jobs
 - Total candidates
 - Candidates currently screening
 - Shortlisted candidates
 - Upcoming interviews
 - Offers
 - Hires
 - Recruitment trends
-

20. Advanced AI Screening

The Track A matching system will be improved in Track B.

Instead of only comparing keywords, we can use:

- Semantic matching
- Embeddings
- Skill normalization
- Experience analysis
- Multiple matching criteria
- ML-based candidate ranking

The official Track B requirements call for sophisticated candidate matching using ML models.

21. LangGraph Recruitment Agent

In Track A, we will use LangChain for the prototype.

In Track B, we will move toward **LangGraph** so that the recruitment process can be handled as a proper multi-step workflow.

For example:

Job Analyzer

↓

Candidate Screening

↓

Candidate Matcher

↓

Candidate Ranker

↓

Assessment

↓

Interview Scheduler

↓

Communication

↓

Analytics

Each step can have its own logic and can pass information to the next step.

22. Assessments

The product should support candidate assessments.

Depending on API availability, we can integrate assessment platforms such as:

- HackerRank
- HackerEarth
- Codility
- Mettl

These platforms are among those identified in the source as relevant assessment systems.

The product could allow recruiters to:

- Assign an assessment
- Track completion
- View scores
- Add assessment results to the candidate profile

23. Interview Management

The product should improve the basic scheduling feature from Track A.

Recruiters should be able to:

- Schedule interviews
- Assign interviewers
- Reschedule
- Track interview status
- Collect feedback
- View interview history

Calendar integration can be added here.

24. Complete Candidate Journey

Each candidate should have a complete history.

Example:

Applied

→ **Screened**

→ **Shortlisted**

→ **Assessment**

→ **Technical Interview**

→ **HR Interview**

→ **Offer**

→ **Hired**

or:

Rejected / Withdrawn / On Hold

This gives recruiters a complete picture of what happened to each candidate.

25. Hiring Team Collaboration

One major difference between the prototype and product will be collaboration.

The recruiter shouldn't be the only person using the system.

For example:

Recruiter

"I shortlisted Candidate A."

Hiring Manager

"I reviewed Candidate A and gave 4/5."

Recruiter

"Let's schedule the technical interview."

This is the kind of workflow we want the product to support.

26. Recruitment Analytics

The product should provide useful information rather than just showing charts for the sake of having charts.

For example:

Hiring Funnel

100 Candidates

↓

40 Screened

↓

15 Shortlisted

↓

8 Interviewed

↓

3 Offers

↓

2 Hired

The system can also track:

- Time-to-hire
- Candidate quality
- Interview-to-offer ratio
- Hiring conversion
- Candidate pipeline
- Hiring manager feedback

The source specifically recommends these kinds of recruitment efficiency measurements.

27. Diversity and Fairness

Since recruitment AI can introduce bias, the product should not blindly make hiring decisions.

The system should focus mainly on job-related factors such as:

- Skills
- Experience
- Education
- Assessments
- Job requirements

We can also provide diversity-related analytics to help recruiters identify potential differences in the recruitment funnel.

The source specifically recommends bias detection, algorithm audits, transparent metrics, and skills-based matching rather than demographic factors.

28. Security and Privacy

In the prototype, we can keep security relatively simple.

In the product, this becomes important.

We need to protect candidate information through:

- User authentication
- Role-based access
- Secure API keys
- Secure database access
- Data protection
- Data deletion
- Input validation
- Privacy-aware logging

The project document specifically highlights encryption, anonymization, secure storage, and candidate data deletion.

29. Product Technology

Frontend

- React
- Next.js
- Tailwind CSS
- Ant Design

Backend

- FastAPI
- Python
- LangGraph
- Celery

Database

- PostgreSQL

Cache

- Redis

AI

- GPT-4 or Claude

Authentication

- Auth0 or Firebase Auth

Deployment

- Vercel for frontend
- Railway/Render for backend

This follows the recommended Track B technology stack in the project document.

30. API Integrations

The product target is **5+ integrations**.

We can choose integrations based on what APIs/access are realistically available.

Possible areas include:

- Recruitment platforms
- Assessment platforms
- Email
- Calendar
- HRMS/ATS
- Background verification

The original document lists Indian platforms such as Naukri RMS, Hrist, CutShort, Keka, Zoho Recruit, HackerEarth, Mettl, AuthBridge, and others as possible recruitment ecosystem integrations.

We should not claim an integration is complete until its API is actually accessible and working.

31. Two-Person Team Responsibilities

Member 1 – AI & Backend

Main focus:

- Resume processing
- AI
- Matching
- Ranking
- LangChain
- LangGraph
- FastAPI
- PostgreSQL
- Redis
- API integrations

- Assessment backend
 - Scheduling backend
 - Analytics backend
 - Security
 - Testing
-

Member 2 – Frontend & Product

Main focus:

- Streamlit prototype
 - React/Next.js product
 - UI/UX
 - Recruiter dashboard
 - Candidate dashboard
 - Job management
 - Candidate pipeline
 - Assessment interface
 - Interview interface
 - Analytics dashboard
 - Authentication interface
 - Hiring manager interface
 - Documentation
 - Demo
-

32. 8-Week Timeline

Week 1

Prototype Foundation

Build:

- GitHub repository
 - Python environment
 - Streamlit
 - SQLite
 - Resume parser
 - Basic candidate model
-

Week 2

Resume Screening

Build:

- Resume extraction
- Skill extraction
- Education extraction
- Experience extraction
- Candidate profile
- Basic screening

Result

First working AI screening demo

Week 3

Matching

Build:

- Job requirement extraction
- Candidate matching
- Candidate score
- Candidate ranking

Result

AI-powered candidate ranking

Week 4

Prototype Complete

Add:

- Candidate pipeline
- Interview scheduling
- Communication templates
- Basic reports
- Deployment

Result

Track A Prototype

At this point, test the prototype with different jobs and resumes.

33. Week 5

Product Architecture

Start converting the prototype into a product.

Backend

- FastAPI
- PostgreSQL
- Redis
- LangGraph

Frontend

- Next.js
- Authentication
- Product design
- Recruiter dashboard

Result

Product Alpha

34. Week 6

Product Features

Add:

- Candidate management
- Advanced matching
- Candidate pipeline
- Assessments
- Interview scheduling
- Hiring manager access

Result

Product Beta

35. Week 7

Integrations and Analytics

Add:

- Recruitment integrations
- Assessment integration
- Calendar integration
- Analytics
- Diversity metrics
- Monitoring
- Security improvements

Result

Near-production product

36. Week 8

Final Product

Focus on:

- Testing
- Bug fixing
- Performance
- Security
- Deployment
- Documentation
- Demo video
- Technical blog
- Presentation

Final Result

Recruiter AI Agent – Production Product

37. What We Will Show in the Final Demo

We should not just show individual features.

We should show one complete recruitment story.

Example

A recruiter has an opening for:

Python Backend Developer

They create the job and upload 10 resumes.

Our system:

1. Reads the resumes
2. Understands the job
3. Extracts candidate skills
4. Matches candidates
5. Ranks them
6. Explains the ranking
7. Shortlists the best candidates
8. Assigns an assessment
9. Schedules an interview
10. Collects feedback
11. Moves candidates through the pipeline
12. Shows recruitment analytics

This will make the project feel like a real product rather than a collection of AI features.

38. What Makes Our Project Different

Our main strength is the progression from prototype to product.

We are not simply building a large system from the beginning.

We are first proving the most important part:

Does AI actually improve candidate screening?

Once that works, we build the rest of the recruitment platform around it.

So the story of our project becomes:

Problem

↓

AI Prototype

↓

Testing & Validation



Improved AI



Production Architecture



Recruitment Product

39. Final Deliverables

Track A Prototype

- Working Streamlit application
- Resume screening
- Job analysis
- Candidate matching
- Candidate ranking
- Candidate database
- Basic scheduling
- Basic reports
- Deployment
- README

Track B Product

- React/Next.js application
- FastAPI backend
- LangGraph agent
- PostgreSQL
- Redis
- Authentication
- Role-based access
- Candidate lifecycle
- Assessments
- Scheduling
- Collaboration
- Analytics
- Diversity insights
- 5+ integrations where feasible
- Security documentation
- API documentation
- Performance benchmarks
- Technical blog
- Production deployment

The original submission requirements also call for the GitHub repository, README, architecture documentation, live deployment, demo video, and viva presentation; Track B adds the technical blog, performance benchmarks/monitoring, security assessment, and API documentation.

40. Final Project Statement

Our project is an AI-powered recruitment platform developed in two stages.

Track A is our prototype, where we validate the core AI capabilities of resume understanding, candidate matching, screening, and ranking.

Track B is our product, where we transform the validated prototype into a complete recruitment platform with multiple users, advanced AI workflows, assessments, interview management, analytics, integrations, security, and scalable architecture.

The overall journey is:

Prototype → Validate → Improve → Productize → Deploy

This approach allows our two-person team to first build something that works and then gradually turn it into something that can realistically be used as a recruitment product.